Learn HTML

Lesson 7: HTML Attributes & Accessibility

HTML Attributes & Accessibility - Study Notes

Key Concepts

1. **HTML attributes** – modifiers that give elements extra information (e.g., `class`, `id`, `title`).
2. **Class vs. ID** – `class` can be reused on many elements; `id` must be unique on a page.
3. **The `title` attribute** – provides advisory text that appears as a tooltip and can aid screen reader users.
4. **ARIA (Accessible Rich Internet Applications)** – a set of attributes that improve accessibility for users of assistive technology.
5. **Semantic markup + ARIA** – using proper HTML elements first, then adding ARIA only when native semantics aren't enough.

Summary

HTML attributes are the building blocks that let you describe, style, and interact with elements on a page. The most common attributes for beginners are `class`, `id`, and `title`. `class` groups elements for CSS styling or JavaScript selection, while `id` uniquely identifies a single element (useful for anchors, form labels, and DOM manipulation). The `title` attribute supplies supplemental information that browsers display as a tooltip and that screen readers can read when configured, helping users understand the purpose of a control.

Accessibility goes beyond visual cues. ARIA attributes (e.g., `role`, `aria-label`, `aria-hidden`, `aria-expanded`) give assistive technologies semantic hints about custom widgets, dynamic content, and interactive states. The best practice is to rely on native HTML semantics first—`<button>`, `<nav>`, `<header>`, etc.—and add ARIA only when native elements cannot convey the required meaning. Simple ARIA usage, combined with thoughtful `class`, `id`, and `title` attributes, makes your pages more usable for everyone.

Important Points

- **`class`**

- Syntax: `<element class="class-name">`
- Multiple classes: `class="btn primary large"`
- Used by CSS (`.class-name {}`) and JavaScript (`document.querySelectorAll('.class-name')`).

- **`id`**

- Syntax: `<element id="unique-id">`
- Must be **unique** within the entire HTML document.
- Helpful for fragment identifiers (`#unique-id`) and `label` `for` connections (`<label for="email">`).

- **`title`**

- Syntax: `<element title="Helpful tooltip text">`
- Appears on hover in most browsers.
- Screen readers may read it, but it should **not replace** proper labeling (`aria-label`, `<label>`, etc.).
    • **Basic ARIA attributes**
- `role` – defines the element's purpose (`role="button"`).
- `aria-label` – provides an accessible name when visible text isn't sufficient.
- `aria-hidden="true"` – hides decorative elements from assistive tech.
- `aria-expanded` – indicates the state of collapsible content (`true` / `false`).
    • **When to use ARIA**
1. You're creating a custom widget that isn't a native HTML element.
2. You need to convey state changes that aren't automatically announced.
3. You must hide non essential content from screen readers.
    • **Best practice hierarchy**
1. Choose the most **semantic HTML element** possible.
2. Add **native attributes** (`type`, `disabled`, `required`, etc.).
3. Add **ARIA** only if native semantics are insufficient.

Examples

Example 1 – Using `class`, `id`, and `title`

```html
<!-- A styled button with a tooltip -->
<button id="submitBtn" class="btn btn-primary" title="Send your form">
  Submit
</button>
```

Explanation

- `id="submitBtn"` lets JavaScript target this exact button (`document.getElementById('submitBtn')`).
- `class="btn btn-primary"` applies two CSS classes for visual styling.
- `title="Send your form"` shows a tooltip on hover and gives extra context for screen reader users.

Example 2 – Adding basic ARIA to a custom dropdown

```html
<div class="custom-select" role="listbox" aria-label="Choose a fruit"
     aria-expanded="false" tabindex="0">
  <div class="selected-option">Select…</div>
  <ul class="options" hidden>
    <li role="option" tabindex="-1">Apple</li>
    <li role="option" tabindex="-1">Banana</li>
    <li role="option" tabindex="-1">Cherry</li>
  </ul>
</div>
```

```

**Explanation**

- `role="listbox"` tells assistive tech that this `<div>` behaves like a listbox.
- `aria-label` supplies an accessible name because there's no visible `<label>`.
- `aria-expanded` reflects whether the options list is open (`true`) or closed (`false`).
- Each option gets `role="option"` so screen readers announce them as selectable items.
- `tabindex` makes the widget focusable and allows keyboard navigation.

---

## Quick Review

1. **What is the main difference between `class` and `id`?**
2. **When should you prefer a native HTML element over adding a `role` attribute?**
3. **How does the `title` attribute help accessibility, and what are its limits?**
4. **Name two ARIA attributes you might use on a collapsible panel and explain their purpose.**
5. **True or False:** An element can have multiple `id` attributes.

---

## Further Reading

- **Semantic HTML** – why using the right element (`<nav>`, `<header>`, `<button>`, etc.) is the first step to accessibility.
- **ARIA Authoring Practices** – detailed patterns for menus, tabs, dialogs, and more.
- **WCAG 2.1 Success Criteria** – understanding the standards that guide accessible design.
- **CSS Selectors & Specificity** – how `class` and `id` affect styling precedence.
- **JavaScript & the DOM** – manipulating elements via `id`, `classList`, and ARIA state attributes.
```